Living Textbooks

A Handbook for authors to use WSL, Quarto, and GitHub Pages to create and maintain Living Textbooks

Sridhar Ramachandran

July 04, 2026

Table of contents

Preface

Welcome to the world of **Living Textbooks**. I believe this is the biggest revolution shift in academic publishing. These are also known as open-access online textbooks. Instead of waiting five years for a publisher to print a costly "6th Edition" just to fix typos and update a few charts, authors now treat their textbooks like software: they update them continuously in real time, and students access them for free online.

This handbook is a guide that will help authors follow the exact modern workflow used by tech and science professors today.

With a living textbook, you as an author can update instantly while students are actively using it.

Key Advantages of a Living Textbook

- Crowdsourced Errata: Many authors put a "Report a Typo" link at the bottom of every page that links directly to their GitHub. If a student spots a broken link or a misspelling, they can submit a fix, and the author can accept it with one click.
- Interactive Elements: Because it's a website, authors can embed interactive Python/R code blocks, videos, or animations directly inside the text—things a paper textbook could never do.

- No "Out of Stock" issues: Every student has access from day one, entirely for free.

This **handbook** serves as a step-by-step documentation manual for setting up a modern, automated technical writing environment using **Windows Subsystem for Linux (WSL)**, **Quarto**, and **GitHub Pages**.

Use **Quarto** to write, **GitHub** to store your files, and **GitHub Pages** to host the website. This handbook is divided into Chapters. Each chapter builds sequentially on the last, providing complete code snippets and dedicated troubleshooting sections covering the exact edge-case failures you might encounter during the setup process.

Version History

Version	Date	Author	Description of Changes
1.0.0	July 04, 2026	Sridhar Ramachandran	Updated deployment chapter, callouts, formatting, added changelog
0.0.3	July 03, 2026	Sridhar Ramachandran	Updated github integration chapter
0.0.2	Jun 28, 2026	Sridhar Ramachandran	Updated figures, appendix
0.0.1	Jun 27, 2026	Sridhar Ramachandran	Initial official publication

1 Introduction

Writing a textbook is a massive undertaking, but choosing the right tooling can save you hundreds of hours of formatting headaches.

I use **VS Code on Windows** via the **Remote - WSL extension** to write books in **Markdown**, and compile it using **Quarto installation on Ubuntu WSL instance** for local building, and push the source files to **GitHub** for version control and use **GitHub Actions** to automatically build, render and host my books on **GitHub pages**.

This technical handbook is an engineering manual that starts with assuming the technical environment, moves linearly through execution and wraps up with the lomg term management and updates - with dedicated troubleshooting sections - as applicable.

1.1 About Quarto

Quarto is best for most authors, especially those including data, code, or math. Quarto is an open-source scientific and technical publishing system backed by Posit (formerly RStudio). It allows you to write in plain Markdown but handles complex LaTeX math perfectly under the hood. More on Quarto, click here Quarto

1.1.1 How it works:

- You write individual chapters in Markdown (.qmd or .md).
- A simple configuration file (_quarto.yml) stitches them together into a beautiful textbook.
- Outputs can be PDF (via LaTeX), HTML/E-book, and MS Word from the exact same source files.

1.1.2 Why it's easy:

- You don't need to know deep LaTeX formatting;
- Quarto handles cross-references (e.g., "See Figure 2.3"), citations, and indexing automatically.

2 System Prerequisites

2.1 The "Living Textbook" Tech Stack

To make this work smoothly, you use three open-source tools together:

- The Writing Tool (Quarto): You write your textbook in simple Markdown.
- The Code Repository (GitHub): You store your textbook files on GitHub.
- The Web Host (GitHub Pages): A free service that takes your Markdown files and turns them into a public website for your students.

Before launching a Quarto project, your local partition requires a Linux kernel layer running alongside your Windows host file system.

2.2 Environment Architecture

- **OS:** Windows 11 with WSL2 (Ubuntu 22.04 LTS or 24.04 LTS kernel). This is the closest to real Linux dev while keeping Windows usability. WSL (Windows Subsystem for Linux) is a feature in Windows that allows you to run a Linux distribution directly within Windows without needing a virtual machine or dual-boot setup. It provides a compatibility layer for executing Linux binaries natively on Windows.
- **IDE:** VS Code connected via the Remote - WSL extension.

2.3 Download Quarto

- Quarto like any software tool (similar to Python or Git) needs to be installed globally on your Ubuntu system so that it can be run from anywhere. On Ubuntu inside WSL, you can install Quarto via your terminal in just a few commands. In your terminal, run these commands:

```
# 1. Download the installer file to your system.
wget https://github.com/quarto-dev/quarto-cli/releases/download/v1.4.555/quarto-1.4.555-linux
# wget just downloads the temporary .deb installer file to whatever folder you are currently

# 2. Install it globally using Ubuntu's package manager.
sudo apt install ./quarto-1.4.555-linux-amd64.deb
# sudo apt install extracts that file and installs Quarto globally into Ubuntu's system folde
# Once the installation is complete, the quarto command becomes available everywhere across y

# 3. Make sure Quarto is alive and kicking
quarto --version
# It should return the version number. You can safely clean up the installer file using the r

# 4. Clean up and delete the installer file since you don't need it anymore
rm quarto-1.4.555-linux-amd64.deb
```

Permission denied Errors

You may see a classic Ubuntu `_apt` permissions quirk! This is a very common side-effect when you download a file using `wget` and then try to hand it off to `apt` immediately. Because `wget` saved the file with permissions restricted to your personal user account, the system user named `_apt` isn't allowed to read it to complete the installation.
The Fix: We can fix this instantly by granting `_apt` permission to read the file, or by using a slightly different command to install it.

```
# Give everyone read permission for just this installer file
chmod +x ./quarto-1.4.555-linux-amd64.deb

# Run the install again using dpkg (which bypasses the _apt user restriction)
sudo dpkg -i ./quarto-1.4.555-linux-amd64.deb

# Fix any potential missing dependencies just in case
sudo apt-get install -f
```

Then continue with # 3 above

Keeping Your System Up to Date

If you already have Quarto installed but need to jump to the latest release to leverage newer background compiling features, follow the step-by-step guideline detailed in Appendix A: Upgrading Quarto.

2.4 Get a free GitHub Account

- If you don't have one already, sign up at GitHub.

3 First-Time Book Project Setup

In this chapter, we will generate a pre-configured textbook template. Quarto makes creating a book incredibly easy because it has a built-in template that sets up the structure for you.

3.1 Initialize the book structure

Make sure you are in your desired directory inside WSL. Run the following commands. These commands will create the textook project right there.

```
# Initialize a new Quarto book project skeleton
quarto create project book my-textbook
```

> **i** You create the book project exactly once.
>
> It acts as the "bookshelf" and container for your entire project. `book` is a specific project type that tells Quarto exactly how to structure and compile your files. If you type `quarto create project book`, Quarto knows you are writing a multi-page document. It automatically sets up features like:
>
> - A sidebar menu for navigating chapters.
> - Continuous page numbering across the whole project.
> - Cross-referencing capabilities (so you can link Section 1 to Section 4 seamlessly).
> - The ability to export everything into a single PDF or EPUB.
>
> Chapters are just standard text files (like intro.qmd or chapter1.qmd) that live inside that book container. You don't need a command to create them; you just make a new text file inside VS Code whenever you want to start a new chapter!

```
# Navigate into the project directory
cd my-textbook

# Initialize local workspace link to VS Code
code .
```

This creates a new folder called my-textbook inside your selected directory with everything you need:

- _quarto.yml: The main configuration file (the "brain" of your book).
- index.qmd: Your textbook's introduction page.
- intro.qmd, summary.qmd: Example chapters.

Open this new folder inside VS Code. You can write standard Markdown in these files, and insert any code by enclosing it in 3 back tick blocks (e.g., `git add .`), and LaTeX math by enclosing it in $ blocks (e.g., $E = mc^2$).

3.2 Clean Up and Configure Files

Inside VS Code, look at the Explorer sidebar. You will see several files. Let's configure them.

3.2.1 Create a `.gitignore`:

Create a new file named .gitignore in the root folder and paste these lines so you don't accidental upload temporary files to GitHub. Following is an example of content in a typical `.gitignore` file

```
/.quarto/
/_book
*.deb
/_site/
/.ipynb_checkpoints/
*.csv
*.xlsx
*.rds
*.pkl
*.mp4

**/*.quarto_ipynb
```

3.2.2 Update Book Settings (`_quarto.yml`)

Open _quarto.yml. This file dictates the layout of your book. Replace its contents with your requirements. Metadata options available are given here. YAML options. Think of the options available as

- Project Metadata - to define project type `book`
- Document Metadata - to define `title`, `author`, `language`, etc.
- Output formats - `html`, `pdf`, `epub`, `docx`
- Lists - multiple values using YAML sequence syntax Supported datatypes include, `string`, `boolean`, `numbers`, `list`, `array`, literal blocks

> **Chapters**
>
> When you have your book outline and chapters decided, open your `_quarto.yml` file in VS Code and replace the chapters: section so it maps out your new book structure cleanly. For instance, you could update the `_quarto.yml` file like this
>
> ```
> book:
>   ...
>   chapters:
>     - index.qmd
>     - 01-prereqs.qmd
>     - 02-project-setup.qmd
>     - 03-github-integration.qmd
>     - 04-deployment.qmd
>     - 05-ongoing-management.qmd
> ...
> ```
>
> Make sure to generate those blank `.qmd` files in your folder so Quarto doesn't throw a "file not found" error. Run this single command block in your terminal to create all five chapter files instantly
>
> ```
> touch 01-prereqs.qmd 02-project-setup.qmd 03-github-integration.qmd 04-deployment.qmd 05-or
> ```

3.2.3 Writing Content

Populate each of the chapters with content, code, gotchas. Open index.qmd (this is your book's homepage / Preface). Replace the text with a quick welcome. Try out writing `code` and $LaTex$ formulae and other `markdown` syntax

> **Writing New Chapters**
>
> Structure the _quarto.yml file to organize your textbook's chapters, sections, and appendix

3.2.4 Preview Content

One of the best features of using Quarto inside VS Code is the live preview. You don't have to keep publishing to the web just to see how your textbook looks while you write. In your VS Code terminal inside your book's directory, run

```
quarto preview
```

Quarto will start a small local development server. It will open a browser window (passing it through seamlessly from WSL to Windows) showing your textbook.

Every single time you press Ctrl + S to save a chapter in VS Code, Quarto notices the change, re-renders that specific page, and instantly refreshes the browser view. You can literally watch your text formatting and LaTeX equations render perfectly side-by-side as you type.

> ⚠ Gotcha: Headless Browser Errors
>
> Because WSL runs as a headless Linux server inside Windows, commands that attempt to automatically open web windows (like `xdg-open`) will fail with: `xdg-open: no method available for opening 'http://localhost:xxxx/'`
> **The Fix:** The engine is running perfectly despite the message. Copy the local loopback string (`http://localhost:xxxx/`) manually, open your standard Windows browser (Chrome/Edge), and paste it into the address bar.

4 Backup Book Content to GitHub

4.1 Create a GitHub Repository:

1. Go to [GitHub]{https://github.com/} in your browser
2. Name your repository exactly my-textbook.
3. Keep it Public so students can see it.
4. Do not add a `README` or `.gitignore` file or `license` yet.
5. Click Create repository

4.2 Push your code to GitHub:

In your VS Code terminal, link your local folder to GitHub and push your files up:

```
git init

# Tell Git, "Hey, look at all these raw .qmd files I edited today. Get them ready to pack awa
git add .

# Puts a timestamped lock and label on that package. It saves the snapshot locally on your ma
git commit -m "Initial book setup"
git branch -M main
```

> **Stop Quarto preview, if running**
>
> `quarto preview` relies on your laptop to run a temporary server. Stop your Quarto preview if it's still running (Ctrl + C) before you save to git.

4.3 Link WSL folder to GitHub

Look at the GitHub page you left open in your browser. Under the heading "...or push an existing repository from the command line", you will see a command to add a remote origin.

Copy and paste your specific command into your WSL terminal. It will look like this (replace with your actual GitHub username)

```
# COPY AND PASTE THE EXACT REMOTE URL FROM YOUR GITHUB PAGE
git remote add origin https://github.com/YOUR-USERNAME/my-textbook.git

# Sends that locked, labeled package out of your laptop and up into your secure cloud backup
git push -u origin main
```

GitHub Credentials

If GitHub prompts you for your credentials, enter your GitHub username and personal access token. GitHub disabled password authentication for Git operations on the command line quite a while ago to protect user accounts. Instead of your regular password, GitHub requires a Personal Access Token (PAT) or an SSH Key. In WSL, generating a PAT is the fastest and easiest way to clear this hurdle. See Appendix C: Generating Personal Access Token (PAT) on GitHub for detail steps.

Go back to your WSL terminal and try to push your files again

```
git push -u origin main
```

- When it asks for your Username, type your regular GitHub username.
- When it asks for your Password, paste your Personal Access Token instead of your password. (Note: Linux terminals hide passwords as you type them, so it will look like nothing is happening when you paste. Just paste it once and hit Enter).

Caching the credentials

Since typing that long token manually every single time is an absolute pain, we can tell Git to safely remember your token on your Ubuntu partition forever. Run this command in your WSL terminal. This tells Git to save your credentials in memory so you don't have to keep pasting the token

```
# Tell Git to Remember Your Token
git config --global credential.helper store

# To save the token into the store, run a quick manual push command
git push origin main

# Enter your username
# Paste your Personal Access Token when it prompts for the password
```

If it says Everything up-to-date, that means Git successfully validated your token and has securely written it to your local WSL disk. From this exact millisecond forward, Git will never ask you for your password in WSL again

5 Publish Book using GitHub Pages

5.1 Github Pages

To turn these files into a website your students can read, we will use GitHub Pages.

5.2 Add the magic automation step:

Quarto has a built-in tool that handles all the heavy lifting of publishing. In your terminal, simply type:

```
quarto publish gh-pages
```

Quarto will ask you to confirm if you want to publish using GitHub Pages. Hit Y (Yes) and press Enter. This command seamlessly compiles all your Markdown chapters into optimized (HTML, CSS, and JavaScript) web pages, creates a hidden deployment branch on GitHub specifically for hosting, upload the HTML, and give you a live link at the very end of the terminal output and pushes the website live. `quarto publish` hands the keys over to GitHub. GitHub hosts your textbook on their massive, 24/7 data centers. Once this step finishes, Quarto will give you a live URL that looks like this: https://YOUR-USERNAME.github.io/my-textbook/

Wait about 30 seconds for GitHub's cache to cycle, and refresh your live link.

Once that publish command finishes and says it's done, your textbook lives permanently on the internet. It stays up and readable for anyone with the link, completely independent of whatever you are doing on your machine.

Give that link to anyone, and they will see a completely responsive, modern online textbook. Whenever you make changes in the future, you fix a typo, add a chapter, or change an assignment in the future, you just type `quarto publish gh-pages` in your terminal again, and the website updates instantly.

> ⚠ Handling deployment crash
>
> If you see error like this while publishing -

```
fatal: a branch named 'gh-pages' already exists
Already on 'main'
Your branch is up to date with 'origin/main'.
```

If the first deployment attempt crashed halfway through during the password check, it leaves behind a half-baked, broken `gh-pages` branch on your local machine which blocks Quarto from creating a clean one
The Fix - We just need to clear out that broken background branch so Quarto can rebuild it cleanly. Run these two commands in your terminal to force-delete the broken branch and retry

```
# Delete the local broken background branch
git branch -D gh-pages

# Tell Quarto to try building it completely from scratch
quarto publish gh-pages --no-prompt
```

Using the –no-prompt flag tells Quarto, "Hey, use the Git credentials I just saved in Step 1, don't stop to ask questions, just deploy."

⚠ Error: Chrome not found

When you run `quarto publish`, you may see an error "Chrome not found". When you run `quarto preview`, Quarto doesn't need a browser installed on Linux because it just builds a tiny local web server and lets your Windows browser do the heavy lifting. However, when you run `quarto publish`, Quarto tries to spin up a hidden background browser (like Chrome or Chromium) inside the Linux environment to handle automated pre-rendering tasks (like generating search indexes, validating links, or rendering complex math equations cleanly). Because your WSL Ubuntu layer is fresh, it doesn't have a Linux browser installed yet.
The Fix: In your WSL terminal, run command to let Quarto download its own isolated, lightweight version of Chromium directly into your toolset.

```
quarto tools install chromium
```

⚠ WARNING: chromium can't be installed fully on WSL with Quarto as system requirements could be missing

`quarto tools install chromium` may lead to this error The legacy quarto tools install chromium relies on a heavy background architecture (Puppeteer) that often misses core

system libraries on Windows Subsystem for Linux and so you may see a warning - "WARNING: chromium can't be installed fully on WSL with Quarto as system requirements could be missing." Quarto actually introduced a much smaller, dedicated tool specifically to handle this without dragging in heavy dependencies or requiring you to mess around with system requirements. This drops in Google's official, lightweight Chrome Headless Shell. It is designed strictly for background compilation, requires almost zero system dependencies, and is built exactly to handle headless platforms like WSL or Docker containers.

```
quarto install chrome-headless-shell
```

⚠ Could not install 'chrome-headless-shell'

If your version of Quarto-CLI does not support `chrome-headless-shell`, try the following. Please review additional notes on Quarto Upgrade to upgrade Quarto. Once this script finishes and you are running the modern version, you can completely ditch those complex browser troubleshooting workarounds we did earlier! The new version natively supports Google's lightweight testing engine. Then try this command

```
quarto install chrome-headless-shell
```

Alternately, we can tell Quarto to compile the project without looking for a browser, and then use the –no-render flag to ship it straight to the cloud.

```
# 1. Force render your book pages locally without hitting the publishing browser gate
quarto render --to html

# 2. Tell Quarto to push the existing build directly to GitHub Pages without re-running the
quarto publish gh-pages --no-render --no-browser
```

`quarto render` handles the conversion of your .qmd files into clean, readable HTML books on your machine without triggering the browser requirements. Specifying `--to html` explicitly tells the compiler to stitch all the chapters tightly into the main sidebar grid configuration before saving the output. `quarto publish --no-render` acts as a pure delivery truck. It tells Quarto, "The book is already built. Don't touch Chrome, don't run a compile step—just grab the files and drop them off at my GitHub repository right now." The `--no-browser` flag tells Quarto: "Just compile my markdown and push the HTML files directly to GitHub Pages. Don't worry about trying to open a browser window on my machine when you're done."

6 Long-Term Maintenance & Ongoing Management

Your technical textbook is a living document. It will require updates, errata fixes, and new content periodically — if not daily.

6.1 How the Continuous Update Workflow Works

Once you set this up, updating your textbook takes seconds. The workflow feels invisible:

1. Fix a typo or add a section:In your local text editor. Open your Markdown chapter file on your computer. Make your edits, add a new LaTeX equation, or drop in a new code example.
2. Push changes to GitHub:10 seconds. Save the file and "push" your update to your GitHub repository (just like saving code to the cloud).
3. Automatic background build: 1-2 minutes.An automated script (called a GitHub Action) triggers in the cloud. It automatically runs Quarto, reads your updated Markdown, updates the textbook's table of contents, and recompiles the website.
4. The student's view is instantly updated: Immediate. The next time a student refreshes the website on their phone or laptop, the new paragraph, fix, or assignment is right there. Because your publishing system is fully integrated with Git and Quarto, updating your live book follows a highly structured, predictable workflow.

6.2 The Daily Writing Loop

Mental Model

Whenever you are ready to modify your content, follow this step-by-step developer loop.

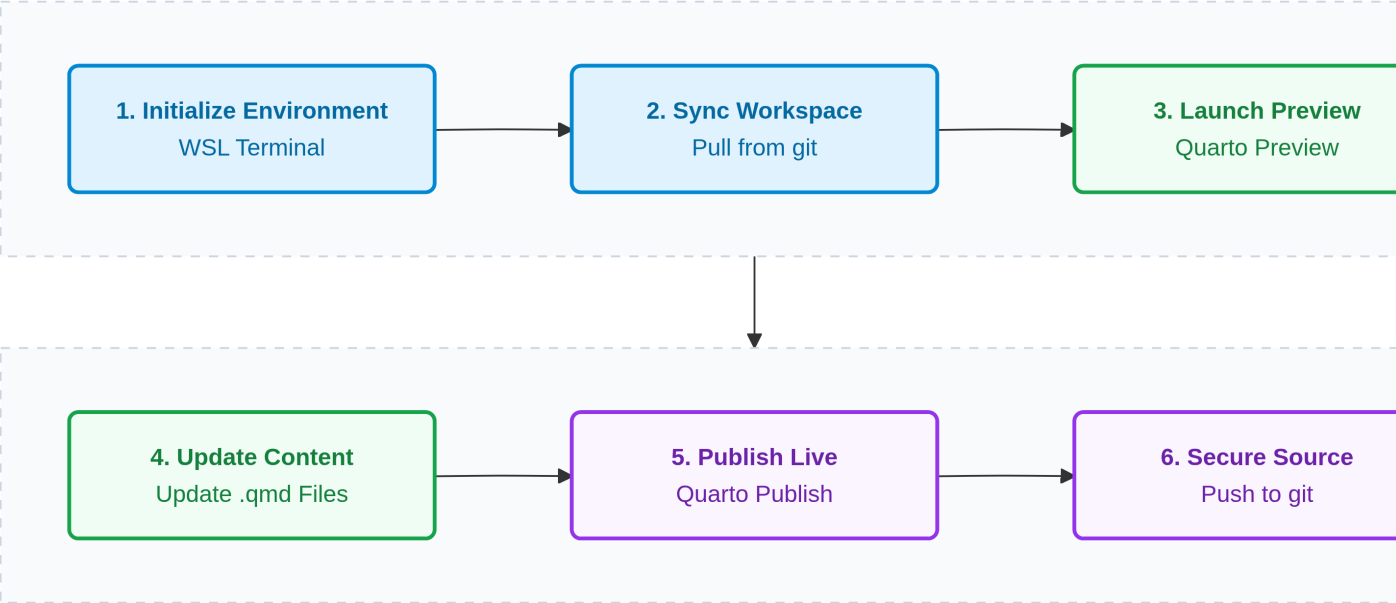

Figure 6.1: Daily Writing and Publishing Workflow

6.2.1 Step 1: Initialize Your Environment

Open your WSL terminal and change directories to your project folder:

```
cd ~/my-textbook
```

6.2.2 Step 2: Sync Your Local Workspace

Before making any edits, always pull the latest tracking tree from your remote repository to ensure you are building on top of your most recent cloud backup:

```
git pull origin main
```

6.2.3 Step 3: Launch Your Live Editing Space

Open the current directory inside VS Code and spin up the local preview daemon. This allows you to view your changes in real-time in your browser as you type:

```
# Open VS Code
code .

# Spin up the local web engine
quarto preview
```

6.2.4 Step 4: Author Your Content

Modify existing .qmd files or generate entirely new ones.

> **!** Important Structural Rule
>
> If you add a brand new chapter file (e.g., 06-advanced-features.qmd), you must declare it inside your _quarto.yml file under the chapters: array. If it is omitted from the configuration mapping, Quarto will not compile it into the final book layout.

6.2.5 Step 5: Publish Your Updates Live

Once your edits look perfect in your local browser preview, it is time to push them live. Halt the local preview engine and run the deployment engine:

Action: Press Ctrl + C in your terminal to stop the preview

```
# Build and push the static production pages to GitHub Pages
quarto publish gh-pages
```

It compiles your raw markdown text into pretty, web-ready HTML pages and uploads them immediately to your live web server. Your readers see your updates instantly.

6.2.6 Step 6: Secure Your Core Source Text

While the production website is now live on GitHub Pages, your raw markdown source files are still only sitting on your local machine. Back up your core workspace changes back to your primary GitHub cloud tree.

Git commands from WSL terminal are provided here for reference.

Appendix: Supplemental Operations

This appendix contains technical maintenance procedures and system modifications that support your core writing environment but fall outside daily authoring workflows.

A. Upgrading the Quarto CLI

Because Quarto is under active, rapid development, you may occasionally need to step your system up to a newer engine baseline. Upgrading inside WSL is highly stable: downloading and running a new installer package automatically replaces the older binary files while leaving your text projects, folders, and workspace variables completely untouched.

> **Note**
>
> When I was working on quarto 1.4.555 version, I used to be asked to login to polyfill.io because the Quarto page relied on an outdated template or MathJax script that still attempted to load a script from the compromised domain polyfill.io. Earlier versions of Quarto (prior to 1.4.557) generated HTML files utilizing MathJax defaults that included links to the polyfill.io.
> The easiest and most effective fix was to download the latest Quarto version, which automatically replaces the vulnerable script with safe alternatives like Cloudflare.
> Another bonus of upgrading: Once this script finishes and you are running the modern version, you will not need those complex browser troubleshooting workarounds! The new version natively supports Google's lightweight testing engine.

The Upgrade Workflow

To pull down and apply the current stable production build, execute this sequence in your WSL terminal:

```
# 1. Purge any stale, leftover installer packages from your environment
rm -f quarto-*-linux-amd64.deb

# 2. Securely fetch the targeted release build from the official core repository
```

```
wget [https://github.com/quarto-dev/quarto-cli/releases/download/v1.9.38/quarto-1.9.38-linux

# 3. Apply the upgrade using Ubuntu's package deployment engine
sudo dpkg -i quarto-1.9.38-linux-amd64.deb

# 4. Confirm the underlying system kernel successfully registered the new version
quarto --version

# 5. Remove downloaded .deb installer
rm -f quarto-*-linux-amd64.deb
```

Git Push issues seen after an upgrade

Based on which folder you downloaded the installer, you may have 'inadvertantly' downloaded it in one of the folders that is included in `git add .`. If that happens this large `.deb` file gets included in the list to be committed to git. The installer file is a large file and since `git push` has a 100MB limit, you will face commit and push issues. This is a classic Git trap! Git captured that massive installer file and tried to track it. Even if you delete the file now using your standard file manager, Git keeps it locked inside its historical staging memory. We can fix this instantly by telling Git to forget the installer ever existed.

```
git rm --cached quarto-1.9.38-linux-amd64.deb
```

Also remember to add `*.deb` to the `.gitignore` file, so that such installer files are not picked by `git add .` Then attempt to add, commit and push again.
At this stage if you still face push error, while .deb is not in the latest commit, it is still trapped inside your local Git history from the previous commit attempt. When you run git push, Git tries to upload the entire historical chain—including that heavy, blocked commit. Easy and cleanest way at this stage would be to simply reset your Git history back to its pristine state before that file got tracked, and then commit your clean files.

```
# 1. Reset your git memory
git reset --mixed origin/main

# 2. Re-stage your clean files
git add .

# 3. Create a fresh clean commit and push to cloud
git commit -m "Re-commit textbook files excluding installer assets"

# 4.
git push origin main
```

B. Common Git commands from WSL Terminal

Backing up source configurations requires connecting your local Linux file directory to a cloud-hosted Git tracking tree.

Connection Commands

Execute these tracking operations inside your root project folder:

```
git init
git add .
git commit -m "initial textbook setup"
git branch -M main
# COPY AND PASTE THE EXACT REMOTE URL FROM YOUR GITHUB PAGE
git remote add origin [https://github.com/YOUR-USERNAME/my-textbook.git](https://github.com/Y
git push -u origin main
```

With that your source files are safely backed up on GitHub

On an ongoing basis to make sure your core markdown files are safely saved to your main GitHub repository cloud, run your standard git backup command every now and then:

```
git add .
git commit -m "added new content"
git push origin main
```

Other useful troubleshooting commands

To verify the last push into git

```
git log -1
```

In this the `-1` indicates the latest version of the pushed files. For the last two commits use -2 and so on.

C. Generating Personal Access Token (PAT) on GitHub

1. Open your browser and go to your GitHub account.
2. Click your profile picture in the top-right corner and go to Settings.
3. Scroll all the way down on the left sidebar and click Developer settings.
4. Click Personal access tokens → Tokens (classic).
5. Click the Generate new token dropdown → select Generate new token (classic).
6. Give it a name (e.g., wsl-textbook).
7. Under Select scopes, check the box for `repo` (this gives it permission to update your repositories).
8. Scroll to the bottom and click Generate token.

> Copy that long token string immediately!
>
> Important: Copy that long token string immediately! Once you navigate away from this page, GitHub will never show it to you again.